

Rapport de synthèse

Attention Is All You Need (Vaswani et al., 2017)

Semi-Supervised Classification with Graph Convolutional Networks (Kipf & Welling, 2017)

Yani Mohellebi

Avril 2026

Résumé

Ce rapport propose une synthèse de deux articles fondamentaux parus en 2017, qui ont chacun marqué leur sous-domaine respectif du *Deep Learning*. Le premier, *Attention Is All You Need* (Vaswani et al., 2017), abolit le recours aux récurrences dans le traitement séquentiel en introduisant l'architecture Transformer. Le second, *Semi-Supervised Classification with GCNs* (Kipf and Welling, 2017), contourne la complexité cubique du filtrage spectral sur les graphes et ouvre la voie au *Geometric Deep Learning* à large échelle. Après une analyse séparée de chaque architecture, de leurs dérivations mathématiques et de leurs résultats expérimentaux, nous mettons en évidence un lien structurel : l'attention peut être vue comme une convolution de graphe opérant sur un graphe complet implicite à arêtes dynamiques, ce qui place les deux modèles dans le cadre commun des *Message Passing Neural Networks* (Gilmer et al., 2017).

Table des matières

1	Introduction	3
2	Synthèse : Attention Is All You Need	3
2.1	Contexte et problématique	3
2.2	Vue d'ensemble du flux de données	3
2.3	Dérivation du Scaled Dot-Product Attention	4
2.3.1	Justification mathématique du facteur d'échelle $\sqrt{d_k}$	4
2.4	Multi-Head Attention	4
2.5	Injection de la séquentialité : encodage positionnel	5
2.6	Composants secondaires indispensables	5
2.7	Encodeur, décodeur et attention masquée	5
2.8	Complexité et résultats	6
3	Synthèse : Semi-Supervised Classification with GCNs	6
3.1	Motivation et positionnement	6
3.2	Fondements en théorie spectrale des graphes	6
3.3	L'approximation de Tchebychev	7
3.4	La dérivation au premier ordre : le cœur du GCN	7
3.4.1	Le « renormalization trick »	7
3.5	Architecture finale et évaluation	8
3.6	Évaluation en apprentissage semi-supervisé	8

4	Corrélation entre Transformer et GCN	8
4.1	Le passage de messages (MPNN)	8
4.2	La corrélation algorithmique	9
4.2.1	Le mécanisme d'adjacence : statique vs dynamique	9
4.3	Conséquences topologiques	9
4.4	Héritage et fusion : les Graph Attention Networks (GAT)	9
5	Conclusion générale	10

1 Introduction

L'année 2017 représente un point de bascule dans l'histoire récente du *Deep Learning*.

D'une part, le traitement automatique du langage naturel (NLP) reposait quasi exclusivement sur les réseaux de neurones récurrents (RNN) et leurs variantes *gated* (LSTM, GRU). Ces modèles traitaient l'information de manière strictement séquentielle, l'état caché h_t à l'instant t dépendant de h_{t-1} . Cette construction posait deux problèmes sérieux. D'abord, l'impossibilité de paralléliser les calculs sur la dimension temporelle : pour calculer h_T , il fallait avoir calculé h_{T-1} , ce qui limitait fortement la vitesse d'itération sur GPU. Ensuite, une dégradation de la capacité à modéliser des dépendances à très long terme en raison de la dilution du gradient temporel (*bottleneck* informationnel) (Bahdanau et al., 2015; Luong et al., 2015).

D'autre part, le domaine de la *Geometric Deep Learning* cherchait à étendre les réseaux de neurones convolutifs (CNN), limités jusque-là aux grilles régulières comme les images, à des structures non euclidiennes : les graphes, les variétés, les nuages de points. L'approche spectrale consistait à transposer la convolution dans le domaine fréquentiel en utilisant la décomposition spectrale du Laplacien du graphe. Et ça exigeait une complexité cubique $\mathcal{O}(N^3)$, d'où l'impossibilité de passer à l'échelle sur des graphes de l'ordre du million de nœuds.

Ce rapport est devisé en trois parties. On retrouve dans sections 2 et 3 une analyse indépendante de chaque article : architecture, dérivation mathématique, évaluation empirique. Tandis que la section 4 montre la relation entre les deux architectures et ce que ça a apporté au monde scientifique.

2 Synthèse : Attention Is All You Need

2.1 Contexte et problématique

La proposition de Vaswani et al. (2017) tient en son titre : l'attention est tout ce dont on a besoin. L'architecture Transformer repose sur l'utilisation d'un mécanisme de *self-attention* permettant une connectivité globale immédiate entre tous les éléments de la séquence, réduisant la distance du chemin d'interaction à $\mathcal{O}(1)$ au lieu de $\mathcal{O}(n)$ dans un RNN.

2.2 Vue d'ensemble du flux de données

Le traitement de l'information au sein du modèle s'articule en quatre phases principales :

- **Embeddings** : les *tokens* d'entrée sont transformés en vecteurs de dimension $d_{\text{model}} = 512$ (*embeddings*), auxquels est additionné un encodage positionnel.
- **Encodage** : cette séquence vectorielle est traitée séquentiellement par $N = 6$ blocs d'encodeur à dimension constante.
- **Décodage** : la séquence cible subit un traitement similaire. Ses $N = 6$ couches intègrent un mécanisme supplémentaire de *cross-attention* pour exploiter les caractéristiques extraites par l'encodeur.
- **Prédiction** : la sortie du décodeur est projetée linéairement vers la dimension du vocabulaire, puis normalisée par un *softmax*. La distribution obtenue permet d'évaluer l'entropie croisée (entraînement) ou de prédire le prochain *token* (inférence).

2.3 Dérivation du Scaled Dot-Product Attention

Le mécanisme central du modèle est le *Scaled Dot-Product Attention*. Il prend en entrée trois objets : une matrice de requêtes (*Query*) Q , une matrice de clés (*Key*) K , et une matrice de valeurs (*Value*) V . Pour chaque position de requête, on calcule une similarité (par produit scalaire) avec toutes les positions de clé, on normalise ces similarités par une fonction *softmax* pour obtenir une distribution de probabilité, puis on agrège les valeurs pondérées par cette distribution.

Avec $Q \in \mathbb{R}^{N \times d_k}$, $K \in \mathbb{R}^{M \times d_k}$ et $V \in \mathbb{R}^{M \times d_v}$, où N et M sont les longueurs des séquences et d_k , d_v les dimensions de l'espace latent, l'équation canonique de l'attention s'écrit :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (1)$$

2.3.1 Justification mathématique du facteur d'échelle $\sqrt{d_k}$

Une contribution théorique clé de l'article (Vaswani et al., 2017) est l'introduction du facteur de mise à l'échelle $1/\sqrt{d_k}$. Les auteurs justifient ce choix par une analyse de variance des produits scalaires en grande dimension. Supposons que les composantes des vecteurs q (requête) et k (clé) soient des variables aléatoires indépendantes de moyenne 0 et de variance 1. Leur produit scalaire $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ a pour moyenne :

$$\mathbb{E}[q \cdot k] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0. \quad (2)$$

Cependant, la variance subit une accumulation :

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k. \quad (3)$$

Ainsi, pour un d_k élevé (par exemple 64), les valeurs du produit scalaire ont une grande variance, poussant la fonction $\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$ dans des régions de saturation où son gradient s'approche de zéro (*vanishing gradient*). La division par $\sqrt{d_k}$ ramène la variance à l'unité, stabilisant l'apprentissage.

2.4 Multi-Head Attention

Plutôt que d'appliquer une fonction d'attention unique sur des vecteurs de dimension d_{model} , Vaswani et al. (2017) proposent de projeter linéairement les requêtes, clés et valeurs h fois, à l'aide de différentes matrices de poids apprises :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (4)$$

où $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Les matrices de projection sont $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ et $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. En pratique, $d_k = d_v = d_{\text{model}}/h$. Cette séparation permet au modèle de prêter simultanément attention à différentes caractéristiques sémantiques (syntaxe, coréférence, temps) dans différents sous-espaces de représentation.

L'architecture (*base model*) emploie une dimension $d_{\text{model}} = 512$ et $h = 8$ têtes d'attention, et ça résulte une dimension $d_k = d_v = 64$ pour chaque sous-espace. Chaque tête évalue ainsi l'attention indépendamment dans un espace vectoriel de dimension 64. Ces huit opérations s'exécutent en parallèle, et leurs sorties sont ensuite concaténées pour reconstituer un tenseur de dimension globale $8 \times 64 = 512$.

2.5 Injection de la séquentialité : encodage positionnel

Puisque le modèle ne contient aucune opération séquentielle de traitement, il est intrinsèquement *invariant par permutation* (*permutation-invariant*). Pour que le modèle puisse appréhender l'ordre des mots, un **encodage positionnel** déterministe est additionné à l'*embedding* d'entrée. Pour une position pos et une dimension i données, cet encodage emploie des paires de fonctions sinus et cosinus à fréquences décroissantes :

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (5)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (6)$$

Cette formulation garantit que pour un décalage donné k , PE_{pos+k} peut s'exprimer comme une fonction linéaire de PE_{pos} , permettant ainsi au modèle d'apprendre facilement l'attention relative.

2.6 Composants secondaires indispensables

Bien que l'attention soit au cœur de l'architecture Transformer, trois composants auxiliaires sont cruciaux pour garantir sa convergence :

- **Connexions résiduelles** (*skip connections*) : pour pallier le problème de la disparition du gradient, chaque sous-couche (attention ou *feed-forward*) s'accompagne d'une connexion résiduelle. La sortie formelle devient $x + \text{Sublayer}(x)$.
- **Normalisation par couche** (*layer normalization*) : elle stabilise la distribution des activations internes. Cela autorise des taux d'apprentissage plus élevés et facilite l'optimisation.
- **Réseau *feed-forward* positionnel** (*position-wise FFN*) : l'attention étant une opération purement linéaire sur les valeurs V , elle manque d'expressivité non linéaire. Pour y remédier, un perceptron multicouche (MLP) est appliqué de manière identique et indépendante à chaque position de la séquence :

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

2.7 Encodeur, décodeur et attention masquée

L'architecture complète pour la traduction comprend deux piles symétriques de couches. Le décodeur diffère de l'encodeur par deux spécificités importantes. D'abord, la *self-attention masquée* : un *token* de position t ne peut attendre que des positions $\leq t$, jamais des positions ultérieures. Ce masquage est réalisé en mettant à $-\infty$ les entrées correspondantes de la matrice $QK^\top$ avant le *softmax*, ce qui produit des poids nuls sur les positions futures. Cette contrainte est indispensable pour que la génération de texte soit autorégressive : à l'inférence, on génère les *tokens* un par un, et chaque nouveau *token* ne doit dépendre que de ce qui le précède.

Ensuite, la *cross-attention* : dans cette sous-couche, les requêtes proviennent du décodeur (de la sortie de la *self-attention* masquée), mais les clés et valeurs proviennent de la sortie de l'encodeur. C'est ce mécanisme qui réalise le pont information source $\rightarrow$ cible d'une traduction : à chaque étape de la génération, le décodeur peut consulter l'intégralité de la phrase source encodée pour décider du mot suivant.

2.8 Complexité et résultats

La complexité de l’attention par couche est de $\mathcal{O}(N^2 \cdot d_{\text{model}})$, où N est la longueur de la séquence. Bien que quadratique en termes de mémoire, le nombre de calculs purement séquentiels est ramené à $\mathcal{O}(1)$. Les résultats expérimentaux ont battu l’état de l’art sur les tâches de traduction WMT 2014 anglais-allemand tout en réduisant le temps d’entraînement à une fraction de celui des architectures CNN/RNN concurrentes.

Sur la tâche de traduction WMT 2014 anglais-allemand, le Transformer de base atteint un score BLEU de 27,3, et le modèle *large* 28,4, dépassant les meilleurs résultats publiés à l’époque (incluant des modèles d’ensemble) comme le ConvS2S (25,2 BLEU) (Vaswani et al., 2017).

3 Synthèse : Semi-Supervised Classification with GCNs

3.1 Motivation et positionnement

Le problème abordé par Kipf and Welling (2017) est celui de la classification de nœuds dans un graphe, dans un régime semi-supervisé : on dispose d’un graphe complet où seuls quelques nœuds par classe sont étiquetés. L’objectif est de propager les étiquettes connues vers les nœuds non étiquetés en exploitant la structure du graphe.

3.2 Fondements en théorie spectrale des graphes

L’article de Kipf and Welling (2017) prend racine dans le domaine fréquentiel des graphes. Soit $G = (V, E)$ un graphe non orienté avec N nœuds, $A \in \mathbb{R}^{N \times N}$ sa matrice d’adjacence et D la matrice diagonale des degrés, définie par $D_{ii} = \sum_j A_{ij}$. Le Laplacien normalisé symétrique du graphe est :

$$L = I_N - D^{-1/2} A D^{-1/2}. \quad (7)$$

Cette matrice joue, sur un graphe, un rôle analogue à celui de l’opérateur de Laplace en analyse classique : elle mesure à quel point un signal varie d’un nœud à ses voisins.

La matrice L étant symétrique semi-définie positive, elle admet une décomposition spectrale :

$$L = U \Lambda U^\top, \quad (8)$$

où les colonnes de U sont les vecteurs propres orthogonaux (qui jouent le rôle de base de Fourier sur le graphe) et Λ est la matrice diagonale des valeurs propres (fréquences). La transformation $\hat{x} = U^\top x$ joue le rôle de *transformée de Fourier sur le graphe*. On retrouve toutes les propriétés structurelles de la transformée de Fourier classique, notamment la possibilité de définir des filtres passe-bas, passe-haut, ou passe-bande sur le graphe en agissant sur le spectre. Une fois cette analogie posée, on peut définir une convolution spectrale : la convolution d’un signal x avec un filtre g_θ s’écrit

$$g_\theta \star x = U g_\theta(\Lambda) U^\top x, \quad (9)$$

où $g_\theta(\Lambda)$ est une fonction du spectre, paramétrée par θ .

Le calcul de la décomposition en valeurs propres U demande $\mathcal{O}(N^3)$ opérations, rendant cette approche inexploitable sur des graphes de grande taille.

3.3 L'approximation de Tchebychev

Pour pallier ce problème de calculabilité cubique, [Defferrard et al. \(2016\)](#) a suggéré d'approximer $g_\theta(\Lambda)$ par une expansion polynomiale de Tchebychev tronquée à l'ordre K :

$$g_{\theta'}(\Lambda) \approx \sum_{k=0}^K \theta'_k T_k(\tilde{\Lambda}), \quad (10)$$

où $\tilde{\Lambda} = \frac{2}{\lambda_{\max}} \Lambda - I_N$. Cette approche permet de localiser le filtre de convolution dans le domaine spatial (rayon de saut K) et d'abaisser la complexité à $\mathcal{O}(K|E|)$, sans devoir calculer U .

3.4 La dérivation au premier ordre : le cœur du GCN

L'idée centrale de [Kipf and Welling \(2017\)](#) est de pousser la simplification à l'extrême en posant $K = 1$ et $\lambda_{\max} \approx 2$ (la borne supérieure théorique du spectre du Laplacien normalisé). Le pari sous-jacent est qu'un empilement de couches simples permet d'obtenir la même expressivité qu'une couche unique à polynômes complexes, et qu'on peut donc se contenter d'un filtre très simple pour chaque couche en compensant par la profondeur.

L'équation devient alors linéaire en le Laplacien :

$$g_{\theta'} \star x \approx \theta'_0 x + \theta'_1 (L - I_N)x = \theta'_0 x - \theta'_1 D^{-1/2} A D^{-1/2} x. \quad (11)$$

Pour limiter encore le nombre de paramètres et réduire le risque de surapprentissage, les auteurs imposent une contrainte supplémentaire : $\theta'_0 = -\theta'_1 = \theta$. Le filtre devient

$$g_\theta \star x \approx \theta \left(I_N + D^{-1/2} A D^{-1/2} \right) x. \quad (12)$$

On voit que ce filtre réalise, intuitivement, une moyenne entre la valeur du nœud lui-même ($I_N x$) et celle de ses voisins ($D^{-1/2} A D^{-1/2} x$, avec une normalisation symétrique par les degrés).

3.4.1 Le « renormalization trick »

Il reste le problème de la stabilité numérique. La matrice $I_N + D^{-1/2} A D^{-1/2}$ a des valeurs propres dans l'intervalle $[0, 2]$. Pour le voir, on part du fait que le spectre du Laplacien normalisé $\mathcal{L} = I_N - D^{-1/2} A D^{-1/2}$ est inclus dans $[0, 2]$ (résultat classique de la théorie spectrale des graphes). Par conséquent, les valeurs propres de $D^{-1/2} A D^{-1/2} = I_N - \mathcal{L}$ sont dans $[-1, 1]$, et celles de $I_N + D^{-1/2} A D^{-1/2}$ dans $[0, 2]$. Une application répétée d'un opérateur dont le rayon spectral atteint 2 provoquerait, dans un réseau profond, une explosion exponentielle des activations, et à l'inverse une disparition pour des opérateurs dont le rayon spectral est inférieur à 1.

La solution proposée par les auteurs consiste à introduire des boucles de retour (*self-loops*) sur le graphe en posant $\tilde{A} = A + I_N$ et $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. La règle de propagation entre deux couches s'écrit alors :

$$H^{(\ell+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(\ell)} W^{(\ell)} \right), \quad (13)$$

où $H^{(\ell)} \in \mathbb{R}^{N \times C_\ell}$ est la matrice des représentations des nœuds à la couche ℓ ($H^{(0)} = X$ étant la matrice des caractéristiques d'entrée), $W^{(\ell)} \in \mathbb{R}^{C_\ell \times C_{\ell+1}}$ la matrice de poids apprise à cette couche, et σ une non-linéarité (en pratique ReLU). On peut vérifier empiriquement et théoriquement que la matrice $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ a un spectre plus stable, ce qui évite les pathologies numériques du filtre initial. La complexité de cette propagation est $\mathcal{O}(|E| \cdot C_\ell C_{\ell+1})$, ce qui explique l'efficacité du modèle même sur de grands graphes.

3.5 Architecture finale et évaluation

Le modèle utilisé dans le papier pour la classification semi-supervisée est volontairement minimaliste : seulement deux couches GCN. La première couche transforme les caractéristiques d'entrée en représentations cachées, la seconde produit les logits de classification :

$$Z = \text{softmax}\left(\hat{A} \text{ReLU}\left(\hat{A} X W^{(0)}\right) W^{(1)}\right), \quad (14)$$

avec $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$. Cette formulation correspond à une opération de « deux sauts » dans le graphe : à la première couche, chaque nœud agrège l'information de ses voisins immédiats ; à la seconde couche, il agrège celle de ses voisins de second ordre.

La fonction de perte est l'entropie croisée, évaluée uniquement sur les nœuds étiquetés :

$$\mathcal{L} = - \sum_{l \in \mathcal{Y}_L} \sum_{f=1}^F Y_{lf} \ln Z_{lf}, \quad (15)$$

où $\mathcal{Y}_L$ est l'ensemble des indices de nœuds étiquetés. C'est ce qu'on appelle un apprentissage *transductif* : le modèle voit l'ensemble du graphe (y compris les nœuds non étiquetés) pendant l'entraînement, mais n'est supervisé que sur la fraction étiquetée. Pendant l'entraînement, les auteurs utilisent l'optimiseur Adam avec un taux d'apprentissage de 0,01, du *dropout* avec un taux $p = 0,5$ appliqué aux représentations entre les couches, et une régularisation L_2 sur les poids de la première couche uniquement. Le modèle est entraîné pendant un maximum de 200 époques avec un *early stopping* basé sur la perte de validation.

3.6 Évaluation en apprentissage semi-supervisé

L'architecture a été testée sur des réseaux de citation (Cora, Citeseer, Pubmed) avec un taux d'étiquetage extrêmement faible (par exemple 20 nœuds par classe, soit environ 5% des données supervisées). Le GCN calcule les représentations pour l'ensemble des nœuds du graphe (transductif), puis la fonction de perte (*cross-entropy*) est évaluée uniquement sur les nœuds disposant de labels. Cette modélisation a dominé les méthodes concurrentes (DeepWalk, Planetoid) avec des sauts de performance de +5 à +10% en précision absolue (Kipf and Welling, 2017).

4 Corrélation entre Transformer et GCN

Le traitement séquentiel du texte et la classification nodale dans les graphes relèvent de paradigmes distincts. Néanmoins, en adoptant une perspective structurelle et topologique, le Transformer de Vaswani et al. (2017) et le GCN de Kipf and Welling (2017) convergent mathématiquement au sein du cadre des *Message Passing Neural Networks* (MPNN) (Gilmer et al., 2017).

4.1 Le passage de messages (MPNN)

Formalisé par Gilmer et al. (2017), le cadre MPNN décrit comment l'information se propage sur une variété non euclidienne. À l'étape t , la mise à jour de la représentation cachée h_i d'un nœud i s'écrit comme une fonction d'agrégation des messages reçus de son voisinage topologique $\mathcal{N}(i)$:

$$m_i^{(t+1)} = \sum_{j \in \mathcal{N}(i)} M_t\left(h_i^{(t)}, h_j^{(t)}, e_{ij}\right) \quad \text{et} \quad h_i^{(t+1)} = U_t\left(h_i^{(t)}, m_i^{(t+1)}\right), \quad (16)$$

où M_t est la fonction de message, U_t la fonction de mise à jour, et e_{ij} l'attribut de l'arête.

4.2 La corrélation algorithmique

L'isomorphisme entre les deux articles peut se formuler comme suit : *l'architecture Transformer est équivalente à l'application d'un GNN sur un graphe complet implicite*.

Si nous traitons une phrase en langage naturel (par exemple de 10 mots) non pas comme une série chronologique, mais comme un ensemble de nœuds virtuels $V = \{m_1, m_2, \dots, m_{10}\}$, où chaque nœud est virtuellement connecté à tous les autres (un graphe complet, $\mathcal{N}(i) = V$), la parenté apparaît.

4.2.1 Le mécanisme d'adjacence : statique vs dynamique

Dans le **GCN** de Kipf and Welling (2017), l'agrégation de voisinage est pondérée par une matrice d'adjacence statique normalisée $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$. Le poids de la relation entre un nœud i et un nœud j est invariable, déterminé *a priori* par la structure empirique du graphe (les arêtes physiques) et leur degré. Le coefficient de pondération scalaire est :

$$c_{ij}^{\text{GCN}} = \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}}. \quad (17)$$

Dans le **Transformer** de Vaswani et al. (2017), l'agrégation des valeurs (les vecteurs V) est pondérée par la matrice d'attention dynamique. La topologie du graphe n'est pas fixe ; elle est inférée à la volée. Le coefficient de pondération, qui sert d'arête virtuelle dans le graphe, est calculé de manière dense par le produit scalaire des représentations latentes des nœuds :

$$c_{ij}^{\text{Transformer}} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right). \quad (18)$$

4.3 Conséquences topologiques

Cette démonstration formelle illustre que l'attention n'est rien d'autre qu'un *attention-based message passing*. Le Transformer calcule pour chaque nœud (mot) sa relation avec l'ensemble du graphe et génère une matrice d'adjacence floue, probabiliste et dense (via le *softmax*) pour propager l'information.

Tandis que le GCN souffre de l'*over-smoothing* (lissage excessif, où tous les nœuds convergent vers la même représentation après plusieurs couches car ils moyennent leurs voisins de manière homogène), le Transformer, grâce à ses arêtes dynamiques basées sur le contenu séquentiel, sélectionne les informations pertinentes et contourne ce problème, bien qu'il paie cette flexibilité par une complexité spatiale quadratique $\mathcal{O}(N^2)$ (la nécessité de calculer toutes les arêtes virtuelles possibles).

4.4 Héritage et fusion : les Graph Attention Networks (GAT)

La corrélation de ces deux travaux fondateurs a directement inspiré l'architecture GAT (Veličković et al., 2018). Le GAT fusionne explicitement ces deux concepts : il opère sur des graphes fixes (comme le GCN) mais, plutôt que d'utiliser la normalisation statique de Kipf and Welling (2017), il utilise le mécanisme d'attention du Transformer pour apprendre les poids des arêtes lors de l'agrégation locale.

5 Conclusion générale

L'analyse montre l'importance qu'ont eue ces deux articles dans le développement de l'IA et l'optimisation des calculs. D'un point de vue scientifique, on peut résumer cela en quelques phrases courtes :

Le **GCN** a permis d'appliquer des théories mathématiques complexes sur de grands graphes. Le **Transformer**, de son côté, a supprimé la contrainte du traitement séquentiel pour permettre au modèle de regarder toute l'information d'un seul coup, ce qui rend les calculs beaucoup plus rapides grâce à la parallélisation.

Au final, ces deux architectures ont ouvert la voie aux nouveaux LLM qu'on connaît aujourd'hui, dont l'héritage se retrouve dans des modèles hybrides comme le GAT ([Veličković et al., 2018](#)).

Références

- D. Bahdanau, K. Cho, and Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*, 2015 (arXiv :1409.0473).
- M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. *NeurIPS*, 2016.
- J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural Message Passing for Quantum Chemistry. *ICML*, 2017.
- T. N. Kipf and M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017.
- M.-T. Luong, H. Pham, and C. D. Manning. Effective Approaches to Attention-based Neural Machine Translation. *EMNLP*, 2015.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *NeurIPS*, 2017.
- P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks. *ICLR*, 2018.